

ASSIGNMENT NO. 10: Window Functions and Analytical Queries

Objective:

The objective of this practical is to help students understand how to perform analytical calculations using window functions in MySQL. Students will learn how to use ROW_NUMBER () and RANK (), divide records into groups using PARTITION BY, arrange rows using the OVER clause, calculate running totals, generate rankings, and solve practical analytical query scenarios

Introduction to Window Functions

A **window function** performs a calculation across a group of related rows while keeping each individual row in the output.

Unlike aggregate functions with GROUP BY, window functions do not combine multiple rows into one summary row. Each original record remains visible along with the calculated analytical value.

Window Function Example

Suppose a sales table contains the following records:

Salesperson	Department	Amount
Aarav	Electronics	50000
Rohan	Electronics	65000
Divya	Electronics	80000

A window function can display each salesperson along with their ranking.

Salesperson	Department	Amount	SalesRank
Divya	Electronics	80000	1
Rohan	Electronics	65000	2
Aarav	Electronics	50000	3

Window Functions and Aggregate Functions

Feature	Aggregate Function with GROUP BY	Window Function
Combines multiple rows	Yes	No
Keeps individual records	No	Yes
Produces summary values	Yes	Yes
Supports ranking	No	Yes
Supports running totals	Limited	Yes

Uses OVER clause	No	Yes
------------------	----	-----

Common Window Functions

Function	Purpose
ROW_NUMBER()	Assigns a unique sequential number to every row
RANK()	Assigns rank while giving the same rank to tied values
DENSE_RANK()	Assigns rank without gaps after tied values
SUM() OVER()	Calculates running or cumulative total
Function	Purpose
AVG() OVER()	Calculates analytical or moving average
COUNT() OVER()	Counts records without grouping them

2. Creating the Sample Database and Sales Table

For this practical, we will use a table named SalesData.

SalesData Table Structure

Column Name	Data Type	Constraint	Description
SaleID	INT	PRIMARY KEY	Unique ID for each sale
Salesperson	VARCHAR(50)	NOT NULL	Name of salesperson
Department	VARCHAR(30)	NOT NULL	Sales department
SaleDate	DATE	NOT NULL	Date of sale
Amount	DECIMAL(10,2)	NOT NULL	Sale amount

MySQL Code to Create Database and Table

```
CREATE DATABASE SalesDB;
USE SalesDB;
CREATE TABLE SalesData (
    SaleID INT PRIMARY KEY,
    Salesperson VARCHAR(50) NOT NULL,
    Department VARCHAR(30) NOT NULL,
    SaleDate DATE NOT NULL,
    Amount DECIMAL(10,2) NOT NULL
);
```

Insert Sample Records

```
INSERT INTO SalesData
(SaleID, Salesperson, Department, SaleDate, Amount)
VALUES
(1, 'Aarav Shah', 'Electronics', '2026-01-05', 50000.00),
(2, 'Priya Patil', 'Furniture', '2026-01-08', 40000.00),
(3, 'Rohan Desai', 'Electronics', '2026-01-12', 65000.00),
(4, 'Sneha Joshi', 'Furniture', '2026-01-15', 55000.00),
(5, 'Karan Mehta', 'Electronics', '2026-02-05', 65000.00),
(6, 'Meera Kulkarni', 'Furniture', '2026-02-10', 30000.00),
(7, 'Neha Shah', 'Clothing', '2026-02-12', 45000.00),
(8, 'Aman Verma', 'Clothing', '2026-02-18', 45000.00),
```

```
(9, 'Divya More', 'Electronics', '2026-03-03', 80000.00),
(10, 'Rahul Sharma', 'Furniture', '2026-03-06', 60000.00),
(11, 'Isha Nair', 'Clothing', '2026-03-10', 70000.00),
(12, 'Om Jadhav', 'Electronics', '2026-03-14', 55000.00);
```

SalesData Table

SaleID	Salesperson	Department	SaleDate	Amount
1	Aarav Shah	Electronics	2026-01-05	50000.00
2	Priya Patil	Furniture	2026-01-08	40000.00
3	Rohan Desai	Electronics	2026-01-12	65000.00
4	Sneha Joshi	Furniture	2026-01-15	55000.00
5	Karan Mehta	Electronics	2026-02-05	65000.00
6	Meera Kulkarni	Furniture	2026-02-10	30000.00
7	Neha Shah	Clothing	2026-02-12	45000.00
8	Aman Verma	Clothing	2026-02-18	45000.00
9	Divya More	Electronics	2026-03-03	80000.00
10	Rahul Sharma	Furniture	2026-03-06	60000.00
SaleID	Salesperson	Department	SaleDate	Amount
11	Isha Nair	Clothing	2026-03-10	70000.00
12	Om Jadhav	Electronics	2026-03-14	55000.00

3. Understanding the OVER Clause

The OVER clause defines the set of rows used by a window function.

It may contain:

Component	Purpose
ORDER BY	Arranges rows for ranking or cumulative calculations
PARTITION BY	Divides rows into separate groups
Window frame	Defines the range of rows included in a calculation

General Syntax

```
window_function() OVER (
PARTITION BY column_name
ORDER BY column_name
);
```

Example: Number All Sales by Amount

```
SELECT
    SaleID,
    Salesperson,
    Department,
    Amount,
    ROW_NUMBER() OVER (
        ORDER BY Amount DESC, SaleID ASC
    ) AS RowNumber
FROM SalesData;
```

Output

SaleID	Salesperson	Department	Amount	RowNumber
9	Divya More	Electronics	80000.00	1
11	Isha Nair	Clothing	70000.00	2
3	Rohan Desai	Electronics	65000.00	3
5	Karan Mehta	Electronics	65000.00	4
10	Rahul Sharma	Furniture	60000.00	5
4	Sneha Joshi	Furniture	55000.00	6
12	Om Jadhav	Electronics	55000.00	7
1	Aarav Shah	Electronics	50000.00	8
7	Neha Shah	Clothing	45000.00	9
8	Aman Verma	Clothing	45000.00	10
2	Priya Patil	Furniture	40000.00	11
6	Meera Kulkarni	Furniture	30000.00	12

Explanation

Query Part	Purpose
ROW_NUMBER()	Generates a unique sequence number
OVER()	Defines the window used by the function
ORDER BY Amount DESC	Places the highest sale first
SaleID ASC	Provides a fixed order when amounts are equal

4. Using ROW_NUMBER()

The ROW_NUMBER() function assigns a unique sequential number to every row. Even when two rows contain the same value, they receive different row numbers.

Syntax

```
ROW_NUMBER() OVER (
    ORDER BY column_name
)
```

Example 1: Assign Row Numbers Based on Sale Amount

```
SELECT
    Salesperson,
    Department,
    Amount,
    ROW_NUMBER() OVER (
        ORDER BY Amount DESC, SaleID ASC
    ) AS SalesRowNumber
FROM SalesData;
```

Sample Output

Salesperson	Department	Amount	SalesRowNumber
-------------	------------	--------	----------------

Divya More	Electronics	80000.00	1
Isha Nair	Clothing	70000.00	2
Rohan Desai	Electronics	65000.00	3
Karan Mehta	Electronics	65000.00	4
Rahul Sharma	Furniture	60000.00	5

Example 2: Find the Top Three Sales

A window function result cannot normally be filtered directly in the same query's WHERE clause. Therefore, the window query is placed inside a subquery.

```

SELECT
    Salesperson,
    Department,
    Amount,
    SalesRowNumber
FROM (
    SELECT
        Salesperson,
        Department,
        Amount,
        ROW_NUMBER() OVER (
            ORDER BY Amount DESC, SaleID ASC
        ) AS SalesRowNumber
    FROM SalesData
) AS RankedSales
WHERE SalesRowNumber <= 3;

```

Output

Salesperson	Department	Amount	SalesRowNumber
Divya More	Electronics	80000.00	1
Isha Nair	Clothing	70000.00	2
Rohan Desai	Electronics	65000.00	3

Uses of ROW_NUMBER()

Scenario	Use
Generate serial numbers	Assign number to every result row
Find top records	Select top 3, top 5, or top 10 records
Remove duplicate records	Identify repeated rows
Page-wise display	Divide records for pagination
Department-wise numbering	Restart numbering for each department

5. Using RANK() and Understanding Ties

The RANK() function assigns a rank to each row according to the specified order.

When two or more rows contain the same value, they receive the same rank. The next rank contains a gap.

Syntax

```
RANK() OVER (
    ORDER BY column_name DESC
)
```

Example: Rank Salespersons by Sale Amount

```
SELECT
    Salesperson,
    Department,
    Amount,
    RANK() OVER (
        ORDER BY Amount DESC
    ) AS SalesRank
FROM SalesData;
```

Output

Salesperson	Department	Amount	SalesRank
Divya More	Electronics	80000.00	1
Isha Nair	Clothing	70000.00	2
Rohan Desai	Electronics	65000.00	3
Karan Mehta	Electronics	65000.00	3
Rahul Sharma	Furniture	60000.00	5
Sneha Joshi	Furniture	55000.00	6
Om Jadhav	Electronics	55000.00	6
Aarav Shah	Electronics	50000.00	8
Neha Shah	Clothing	45000.00	9
Aman Verma	Clothing	45000.00	9
Priya Patil	Furniture	40000.00	11
Meera Kulkarni	Furniture	30000.00	12

ROW_NUMBER() and RANK() Comparison

Amount	ROW_NUMBER()	RANK()
80000	1	1
70000	2	2
65000	3	3
65000	4	3
60000	5	5
55000	6	6
55000	7	6

Difference Between Ranking Functions

Function	Treatment of Equal Values	Rank Gaps
ROW_NUMBER()	Gives different numbers	Not applicable
RANK()	Gives the same rank	Yes

DENSE_RANK()	Gives the same rank	No
--------------	---------------------	----

DENSE_RANK() Example

```
SELECT
    Salesperson,
    Amount,
    DENSE_RANK() OVER (
        ORDER BY Amount DESC
    ) AS DenseSalesRank
FROM SalesData;
```

6. Partitioning Data and Calculating Running Totals Using PARTITION BY

The PARTITION BY clause divides the result into separate groups. The window function is calculated independently inside each group.

Unlike GROUP BY, PARTITION BY does not combine records.

Example 1: Rank Salespersons Within Each Department

```
SELECT
    Salesperson,
    Department,
    Amount,
    RANK() OVER (
        PARTITION BY Department
        ORDER BY Amount DESC
    ) AS DepartmentRank
FROM SalesData
ORDER BY Department, DepartmentRank, SaleID;
```

Output

Salesperson	Department	Amount	DepartmentRank
Isha Nair	Clothing	70000.00	1
Neha Shah	Clothing	45000.00	2
Aman Verma	Clothing	45000.00	2
Divya More	Electronics	80000.00	1
Rohan Desai	Electronics	65000.00	2
Karan Mehta	Electronics	65000.00	2
Om Jadhav	Electronics	55000.00	4
Aarav Shah	Electronics	50000.00	5
Rahul Sharma	Furniture	60000.00	1
Sneha Joshi	Furniture	55000.00	2
Priya Patil	Furniture	40000.00	3
Meera Kulkarni	Furniture	30000.00	4

Explanation

Query Part	Purpose
------------	---------

PARTITION BY Department	Creates a separate group for every department
Query Part	Purpose
ORDER BY Amount DESC	Ranks higher sales first
RANK()	Assigns department-wise rank
Rank restarts	Ranking begins from 1 for every department

Calculating a Running Total

A running total displays the cumulative total from the first row up to the current row.

Example 2: Calculate Overall Running Sales Total

```

SELECT
    SaleID,
    Salesperson,
    SaleDate,
    Amount,
    SUM(Amount) OVER (
        ORDER BY SaleDate, SaleID
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS RunningTotal
FROM SalesData
ORDER BY SaleDate, SaleID;

```

Output

SaleID	Salesperson	SaleDate	Amount	RunningTotal
1	Aarav Shah	2026-01-05	50000.00	50000.00
2	Priya Patil	2026-01-08	40000.00	90000.00
3	Rohan Desai	2026-01-12	65000.00	155000.00
4	Sneha Joshi	2026-01-15	55000.00	210000.00
5	Karan Mehta	2026-02-05	65000.00	275000.00
6	Meera Kulkarni	2026-02-10	30000.00	305000.00
7	Neha Shah	2026-02-12	45000.00	350000.00
8	Aman Verma	2026-02-18	45000.00	395000.00
9	Divya More	2026-03-03	80000.00	475000.00
10	Rahul Sharma	2026-03-06	60000.00	535000.00
11	Isha Nair	2026-03-10	70000.00	605000.00
12	Om Jadhav	2026-03-14	55000.00	660000.00

Running Total Query Components

Component	Meaning
SUM(Amount)	Adds sale amounts
OVER	Applies the calculation as a window function
ORDER BY SaleDate	Calculates total in date order

UNBOUNDED PRECEDING	Starts from the first row
CURRENT ROW	Ends at the current row

Example 3: Department-Wise Running Total

```

SELECT
    Salesperson,
    Department,
    SaleDate,
    Amount,
    SUM(Amount) OVER (
        PARTITION BY Department
        ORDER BY SaleDate, SaleID
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS DepartmentRunningTotal
FROM SalesData
ORDER BY Department, SaleDate, SaleID;

```

Department-Wise Running Total Sample

Salesperson	Department	SaleDate	Amount	DepartmentRunningTotal
Isha Nair	Clothing	2026-03-10	70000.00	160000.00
Divya More	Electronics	2026-03-03	80000.00	260000.00
Om Jadhav	Electronics	2026-03-14	55000.00	315000.00
Rahul Sharma	Furniture	2026-03-06	60000.00	185000.00

7. Practical Analytical Query Scenarios

Window functions are widely used in sales analysis, employee performance, student ranking, financial reporting, inventory management, and business dashboards.

Practical Use Cases

Scenario	Analytical Requirement	Suitable Function
Sales ranking	Rank salespersons by sales amount	RANK()
Top performers	Find top salesperson in each department	ROW_NUMBER()
Cumulative sales	Calculate total sales up to each date	SUM() OVER()
Department comparison	Rank employees department-wise	PARTITION BY with RANK()
Sequential numbering	Assign unique number to every record	ROW_NUMBER()
Performance comparison	Show individual and department average	AVG() OVER()

Scenario 1: Find the Top Salesperson in Each Department

```

SELECT
    Salesperson,
    Department,
    Amount
FROM (
    SELECT
        Salesperson,
        Department,
        Amount,
        ROW_NUMBER() OVER (
            PARTITION BY Department
            ORDER BY Amount DESC, SaleID ASC
        ) AS DepartmentRowNumber
    FROM SalesData
) AS DepartmentSales
WHERE DepartmentRowNumber = 1;

```

Output

Salesperson	Department	Amount
Isha Nair	Clothing	70000.00
Divya More	Electronics	80000.00
Rahul Sharma	Furniture	60000.00

Scenario 2: Display Each Sale with Department Average

```

SELECT
    Salesperson,
    Department,
    Amount,
    AVG(Amount) OVER (
        PARTITION BY Department
    ) AS DepartmentAverage
FROM SalesData
ORDER BY Department, Amount DESC;

```

Department Average Summary

Department	DepartmentAverage
Clothing	53333.33
Electronics	63000.00
Furniture	46250.00

Scenario 3: Find Sales Above Department Average

```

SELECT
    Salesperson,
    Department,
    Amount,
    DepartmentAverage
FROM (
    SELECT

```

```

Salesperson,
Department,
Amount,
AVG(Amount) OVER (
    PARTITION BY Department
) AS DepartmentAverage
FROM SalesData
) AS SalesAnalysis
WHERE Amount > DepartmentAverage;

```

Output

Salesperson	Department	Amount	DepartmentAverage
Isha Nair	Clothing	70000.00	53333.33
Rohan Desai	Electronics	65000.00	63000.00
Karan Mehta	Electronics	65000.00	63000.00
Divya More	Electronics	80000.00	63000.00
Sneha Joshi	Furniture	55000.00	46250.00
Rahul Sharma	Furniture	60000.00	46250.00

Final Summary Table

Concept	MySQL Syntax	Purpose
Sequential numbering	ROW_NUMBER() OVER()	Gives a unique number to every row
Ranking	RANK() OVER()	Assigns ranks with gaps for ties
Dense ranking	DENSE_RANK() OVER()	Assigns ranks without gaps
Partitioning	PARTITION BY	Divides records into analytical groups
Window ordering	ORDER BY inside OVER	Defines calculation order
Concept	MySQL Syntax	Purpose
Running total	SUM() OVER(ORDER BY...)	Calculates cumulative values
Department total	SUM() OVER(PARTITION BY...)	Calculates total within each department
Department average	AVG() OVER(PARTITION BY...)	Calculates group average without grouping rows

Questions

Q1) Create a MySQL database named SalesDB and a table named SalesData. SalesData Table

Column Name	Data Type	Constraint
-------------	-----------	------------

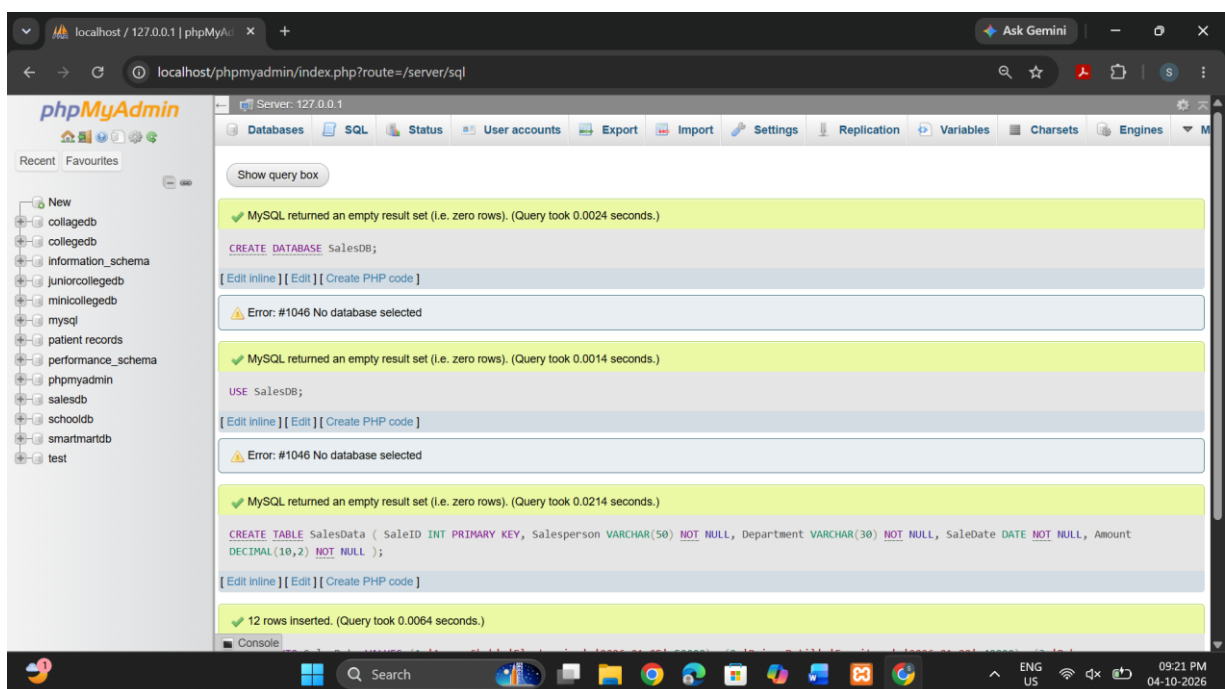
SaleID	INT	PRIMARY KEY
Salesperson	VARCHAR(50)	NOT NULL
Department	VARCHAR(30)	NOT NULL
SaleDate	DATE	NOT NULL
Amount	DECIMAL(10,2)	NOT NULL

Insert at least 12 sales records belonging to three different departments.

```
CREATE DATABASE SalesDB;
USE SalesDB;

CREATE TABLE SalesData (
    SaleID INT PRIMARY KEY,
    Salesperson VARCHAR(50) NOT NULL,
    Department VARCHAR(30) NOT NULL,
    SaleDate DATE NOT NULL,
    Amount DECIMAL(10,2) NOT NULL
);

INSERT INTO SalesData VALUES
(1,'Aarav Shah','Electronics','2026-01-05',50000),
(2,'Priya Patil','Furniture','2026-01-08',40000),
(3,'Rohan Desai','Electronics','2026-01-12',65000),
(4,'Sneha Joshi','Furniture','2026-01-15',55000),
(5,'Karan Mehta','Electronics','2026-02-05',65000),
(6,'Meera Kulkarni','Furniture','2026-02-10',30000),
(7,'Neha Shah','Clothing','2026-02-12',45000),
(8,'Aman Verma','Clothing','2026-02-18',45000),
(9,'Divya More','Electronics','2026-03-03',80000),
(10,'Rahul Sharma','Furniture','2026-03-06',60000),
(11,'Isha Nair','Clothing','2026-03-10',70000),
(12,'Om Jadhav','Electronics','2026-03-14',55000);
```



Q2) Write a MySQL query using ROW_NUMBER() to assign a unique row number to every salesperson.

The result should be arranged by sale amount from highest to lowest.

SELECT Salesperson, Department, Amount,

ROW_NUMBER() OVER (

ORDER BY Amount DESC, SaleID ASC

) AS SalesRowNumber

FROM SalesData;

Required Output

Column
Salesperson
Department
Amount
SalesRowNumber

The screenshot shows the phpMyAdmin interface with the following query executed:

```
SELECT Salesperson, Department, Amount, ROW_NUMBER() OVER (ORDER BY Amount DESC, SaleID ASC) AS SalesRowNumber FROM SalesData;
```

The query results are displayed in a table with the following data:

Salesperson	Department	Amount	SalesRowNumber
Divya More	Electronics	80000.00	1
Isha Nair	Clothing	70000.00	2
Rohan Desai	Electronics	65000.00	3
Karan Mehta	Electronics	65000.00	4
Rahul Sharma	Furniture	60000.00	5
Sneha Joshi	Furniture	55000.00	6
Om Jadhav	Electronics	55000.00	7
Aarav Shah	Electronics	50000.00	8
Neha Shah	Clothing	45000.00	9
Aman Verma	Clothing	45000.00	10
Priya Patil	Furniture	40000.00	11
Meera Kulkarni	Furniture	30000.00	12

Q3) Write a MySQL query using RANK() to rank salespersons according to their sale amount.

Students having the same sale amount should receive the same rank.

SELECT Salesperson, Department, Amount,

RANK() OVER (ORDER BY Amount DESC) AS SalesRank

FROM SalesData;

Server: 127.0.0.1 » Database: salesdb » Table: SalesData

Extra options

Salesperson	Department	Amount	SalesRank
Divya More	Electronics	80000.00	1
Isha Nair	Clothing	70000.00	2
Karan Mehta	Electronics	65000.00	3
Rohan Desai	Electronics	65000.00	3
Rahul Sharma	Furniture	60000.00	5
Om Jadhav	Electronics	55000.00	6
Sneha Joshi	Furniture	55000.00	6
Aarav Shah	Electronics	50000.00	8
Aman Verma	Clothing	45000.00	9
Neha Shah	Clothing	45000.00	9
Priya Patil	Furniture	40000.00	11
Meera Kulkarni	Furniture	30000.00	12

Query results operations

Print Copy to clipboard Export Display chart Create view

Required Output

Column
Salesperson
Department
Amount
SalesRank

Q4) Write a MySQL query to rank salespersons separately within each department.

Use:

```
SELECT Salesperson, Department, Amount,
       RANK() OVER (
         PARTITION BY Department
         ORDER BY Amount DESC
       ) AS DepartmentRank
FROM SalesData;
```

Salesperson	Department	Amount	DepartmentRank
Isha Nair	Clothing	70000.00	1
Aman Verma	Clothing	45000.00	2
Neha Shah	Clothing	45000.00	2
Divya More	Electronics	80000.00	1
Karan Mehta	Electronics	65000.00	2
Rohan Desai	Electronics	65000.00	2
Om Jadhav	Electronics	55000.00	4
Aarav Shah	Electronics	50000.00	5
Rahul Sharma	Furniture	60000.00	1
Sneha Joshi	Furniture	55000.00	2
Priya Patil	Furniture	40000.00	3
Meera Kuikarni	Furniture	30000.00	4

Required Component

RANK()

OVER

PARTITION BY Department

ORDER BY Amount DESC

Q5) Write a MySQL query to calculate the running total of sales according to the sale date.

```
SELECT SaleID, Salesperson, SaleDate, Amount,
       SUM(Amount) OVER (
         ORDER BY SaleDate, SaleID
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS RunningTotal
FROM SalesData
ORDER BY SaleDate, SaleID;
```

SaleID	Salesperson	SaleDate	Amount	RunningTotal
1	Aarav Shah	2026-01-05	50000.00	50000.00
2	Priya Patil	2026-01-08	40000.00	90000.00
3	Rohan Desai	2026-01-12	65000.00	155000.00
4	Sneha Joshi	2026-01-15	55000.00	210000.00
5	Karan Mehta	2026-02-05	65000.00	275000.00
6	Meera Kulkarni	2026-02-10	30000.00	305000.00
7	Neha Shah	2026-02-12	45000.00	350000.00
8	Aman Verma	2026-02-18	45000.00	395000.00
9	Divya More	2026-03-03	80000.00	475000.00
10	Rahul Sharma	2026-03-06	60000.00	535000.00
11	Isha Nair	2026-03-10	70000.00	605000.00
12	Om Jadhav	2026-03-14	55000.00	660000.00

Required Output

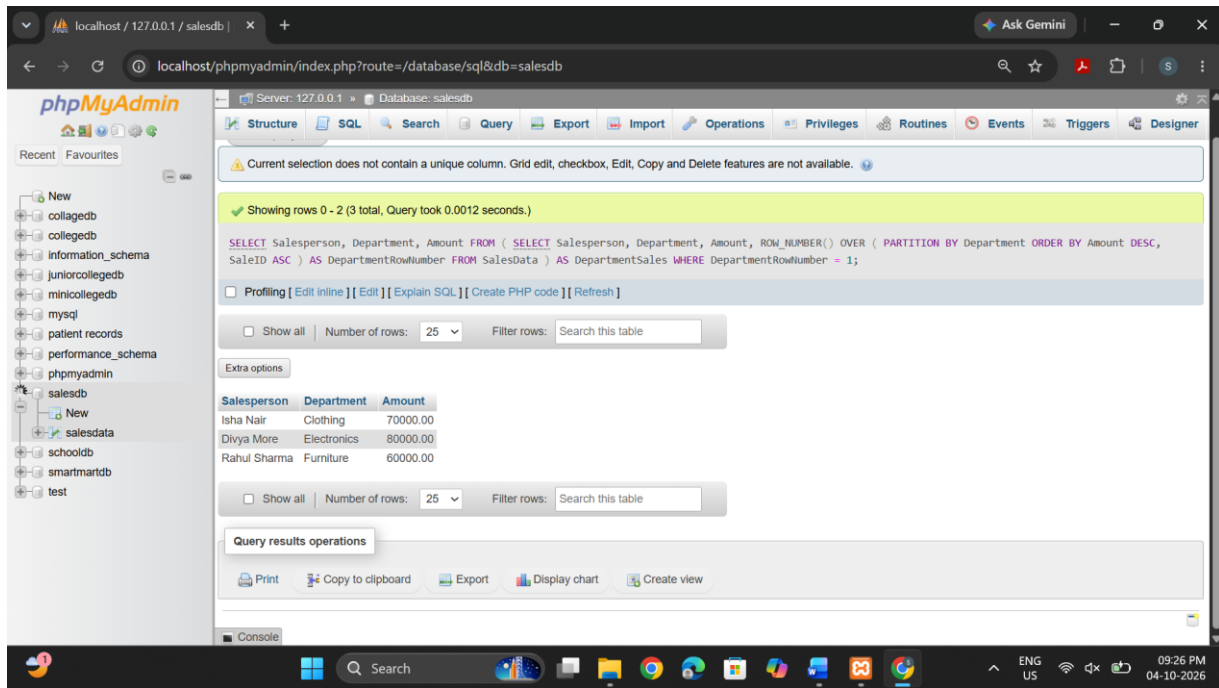
Column
SaleID
Salesperson
SaleDate
Amount
RunningTotal

Q6) Write a MySQL query to display the highest-performing salesperson from each department.

```

SELECT Salesperson, Department, Amount
FROM (
    SELECT Salesperson, Department, Amount,
        ROW_NUMBER() OVER (
            PARTITION BY Department
            ORDER BY Amount DESC, SaleID ASC
        ) AS DepartmentRowNumber
    FROM SalesData
) AS DepartmentSales
WHERE DepartmentRowNumber = 1;

```



The query should:

Requirement
Use ROW_NUMBER()
Partition records by department
Arrange sale amounts in descending order
Use a subquery
Display only the first-ranked salesperson from every department

Assignment Evaluation

- 0: Not Done []
- 1: Incomplete []
- 2: Late Complete []
- 3: Needs Improvement []
- 4: Complete []
- 5: Well Done []

Signature of Instructor: _____